

29. Une méthode : la fonction qui connaît son objet

Ce que fait un ennemi peut s'écrire **DANS** l'ennemi : `troupe[i].avancer()`.

1. Ce qu'il faut comprendre

Une `struct` peut porter des **fonctions** : on les appelle des **méthodes**. Elles s'écrivent **dans** les accolades de la struct, et s'appellent avec un point : `troupe[i].avancer()`;

Dans une méthode, les champs s'écrivent **sous leur nom nu** : `x++`, et non `troupe[i].x++`. La méthode travaille sur **l'objet placé devant le point**. Le code se raccourcit, et surtout il dit **à qui** appartient chaque geste.

Compare avec la leçon précédente : la boucle du jeu devient **trois lignes** — effacer, avancer, dessiner. Tout le détail du mouvement est rangé dans `avancer()`, là où on le cherchera.

Une méthode peut en appeler une autre de **son** objet : `avancer()` appelle `auBord()` sans rien devant — ça veut dire « le mien ».

Sur cette console, une méthode n'a **pas d'arguments** : elle reçoit déjà son objet. Ce qui vient d'ailleurs se passe par une fonction ordinaire.

À toi : ajoute une méthode `dessiner()` qui écrit le X, et une `effacer()`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **618 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
Ennemi::auBord()	29 o	← Ennemi::avancer() — ta fonction
Ennemi::avancer()	104 o	← main() — ta fonction
main()	289 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	16 o	← CopierTuiles — 1 lettre sur 43
" "	1 o	← texte() — un texte gravé
"X"	1 o	← texte() — un texte gravé

Lettres gravées : X

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte() x 2 120 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)

```

image()          x 1    3 o sur place, dans main() + AttendreImage (28 o)
images()         x 1    2 o sur place, dans main()
Tes fonctions appelées :
Ennemi::auBord() x 1    15 o sur place, dans Ennemi::avancer()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : quatre ennemis X font des allers-retours ; cette fois, c'est
// l'ennemi LUI-MÊME qui sait avancer et faire demi-tour.
//
// Ce qui est nouveau : la MÉTHODE, une fonction rangée DANS la struct.
// Dedans, x, y et sens sont ceux de l'ennemi sur qui on l'appelle.
// troupe[2].avancer(); → fait avancer l'ennemi numéro 2, et lui seul.

#include <texte> // écrit un texte à l'écran

```

2. UNE CONSTANTE — COMBIEN

```

const uint8_t COMBIEN = 4; // Le nombre d'ennemis.

```

3. UN ENREGISTREMENT — ENNEMI

```

struct Ennemi { // Le modèle d'un ennemi :
    uint8_t x; // sa colonne,
    uint8_t y; // sa ligne,
    uint8_t sens; // 1 = vers la droite, 0 = vers la gauche.

    uint8_t auBord() { // Méthode : rend 1 si CET ennemi touche un bord.
        return x == 0 || x == 19; // colonne 0 OU colonne 19.
    }

    void avancer() { // Méthode : un pas dans son sens.
        if (sens == 1) x++; // vers la droite,
        else x--; // ou vers la gauche.
        if (auBord()) sens = !sens; // demi-tour : « !sens » change 1 en 0 et 0 en 1.
    }
};

```

4. UN TABLEAU — TROUPE

```

Ennemi troupe[COMBIEN]; // 4 ennemis : troupe[0] à troupe[3].

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() { // Le jeu commence ici.
    for (uint8_t i = 0; i < COMBIEN; i++) { // Pour chaque ennemi i :
        troupe[i].x = 2 + i * 4; // colonne 2, 6, 10, 14
        troupe[i].y = 4 + i * 3; // ligne 4, 7, 10, 13
        troupe[i].sens = 1; // tous partent vers la droite.
    }

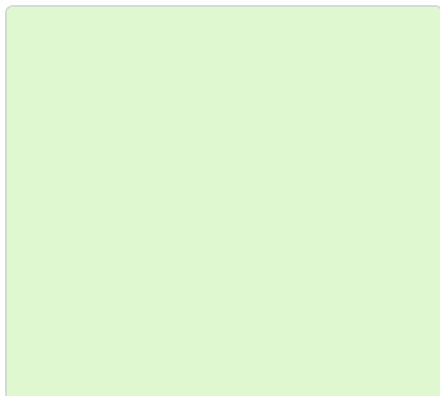
    while (true) { // La boucle du jeu :
        image();
        if (images() % 8 != 0) continue; // on ne bouge qu'une image sur huit.

        for (uint8_t i = 0; i < COMBIEN; i++) {
            texte(troupe[i].x, troupe[i].y, " "); // efface l'ennemi,
            troupe[i].avancer(); // lui demande d'avancer (sa méthode),
            texte(troupe[i].x, troupe[i].y, "X"); // le redessine.
        }
    }
}

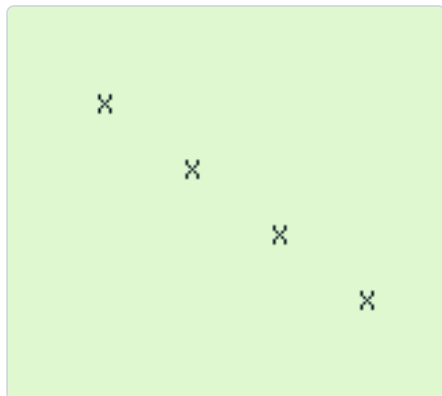
```

3. Ce qui se passe, étape par étape

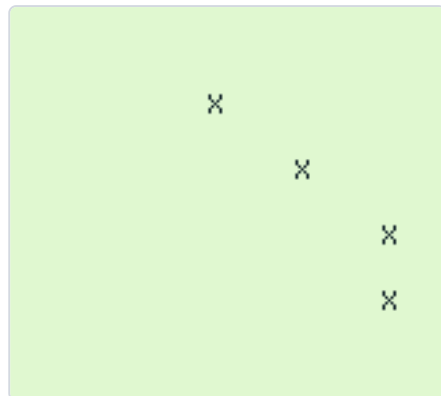
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



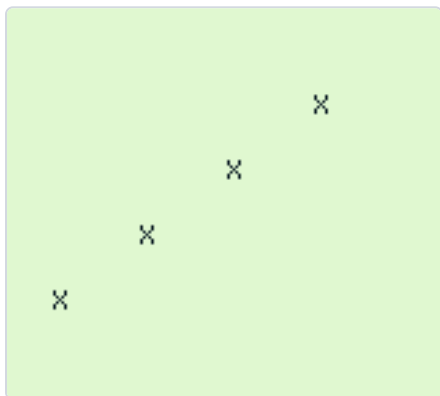
la toute première image — la console vient de démarrer — image 1



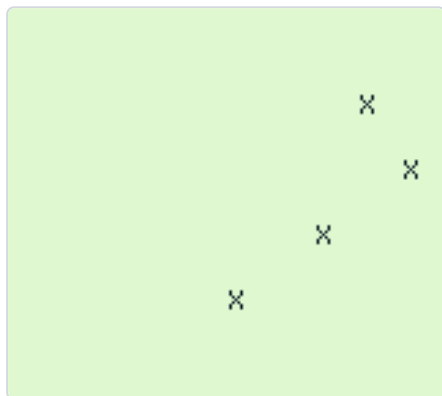
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La même troupe de quatre X qui va et vient — le code du mouvement est rangé dans l'ennemi.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ jamais deux X sur une ligne : l'ancienne case est effacée
- ✓ chacun a parcouru plusieurs cases (20, 20, 20, 20 cases)
- ✓ chacun a touché un bord, et a fait demi-tour

← [28. Un tableau de struct, parcouru par une boucle](#)

[sommaire](#)

[30. Le petit jeu : tout ensemble](#) →